ASSIGNMENT 10

Instructions

- Create a separate **`db.json`** file for each question.
- Include at least 5 valid sample records in every JSON file.
- Run each JSON file on a different port when using JSON Server.
- Use only the JSON file of that specific question for all API operations.

Q1. Real-Time Live Search (jQuery AJAX)

Create a Live Search feature for **`/products`** using **jQuery AJAX**:

- As the user types inside the search box:
    - Send a GET request with query **`?q=value`**.
    - Update the search results below the input instantly.
- Display: Product image, name, price.
- Show *"No products found"* if response is empty.
- Add a loading indicator while waiting for the response.

Q2. Employee Status Dashboard (XMLHttpRequest + PATCH)

Create a dashboard that displays employees with a toggle switch:

- JSON Server **`employees`** has fields: **`id`**, **`name`**, **`status: active/inactive`**.
- Use **XMLHttpRequest** to fetch all employees.
- Each employee row has a toggle button:
    - Active → Inactive
    - Inactive → Active
- On toggle:
    - Send a **PATCH** request using XMLHttpRequest.
    - Immediately update UI to reflect new status.
- If request fails → revert UI and show an error message.

Q3. Task Manager With Filters (jQuery AJAX + Query Params)

Create a Task Manager using **`/tasks`** with fields: **`id`**, **`title`**, **`priority`**, **`completed`**.

Build features:

- **GET /tasks** and show in list format.
- Dropdown for sorting:
    - Low Priority
    - Medium Priority
    - High Priority
    - Completed

      → On change, reload the list using jQuery AJAX with query parameters.
- Add a checkbox on each task:
    - When clicked → Send PATCH request to toggle **`completed`**.

Q4. Multi-API Dashboard (Fetch + Promise.all)

Your dashboard should display:

- Total users
- Total orders
- Total products

All come from different endpoints:

- **`/users`**
- **`/orders`**
- **`/products`**

Requirements:

- Use **Fetch + Promise.all** to load all 3 simultaneously.

- While loading: Show placeholder skeletons.
- If ANY API fails: Show dashboard with warning: *“Some data could not be loaded.”*

Q5. College Timetable Viewer (Fetch + Dynamic Rendering)

Endpoints: **`/timetable?day=Monday`**

Requirements:

- Dropdown for weekdays.
- On change → fetch timetable and render:
  - Subject name
  - Faculty
  - Time
- If no classes: Show message: “No classes today.”

Q6. User Registration With Duplicate Check (Axios + GET + POST)

Create a registration form:

- Before POST:
  - Check if email already exists using GET **`/users?email=abc@gmail.com`**.
- If exists → Show: “Email already registered.”
- Otherwise → Submit using Axios POST.